

Core Concepts — The Layered Reference Architecture

Part 10 — AI Architecture · 10-ai-architecture/notes/01-reference-architecture.md

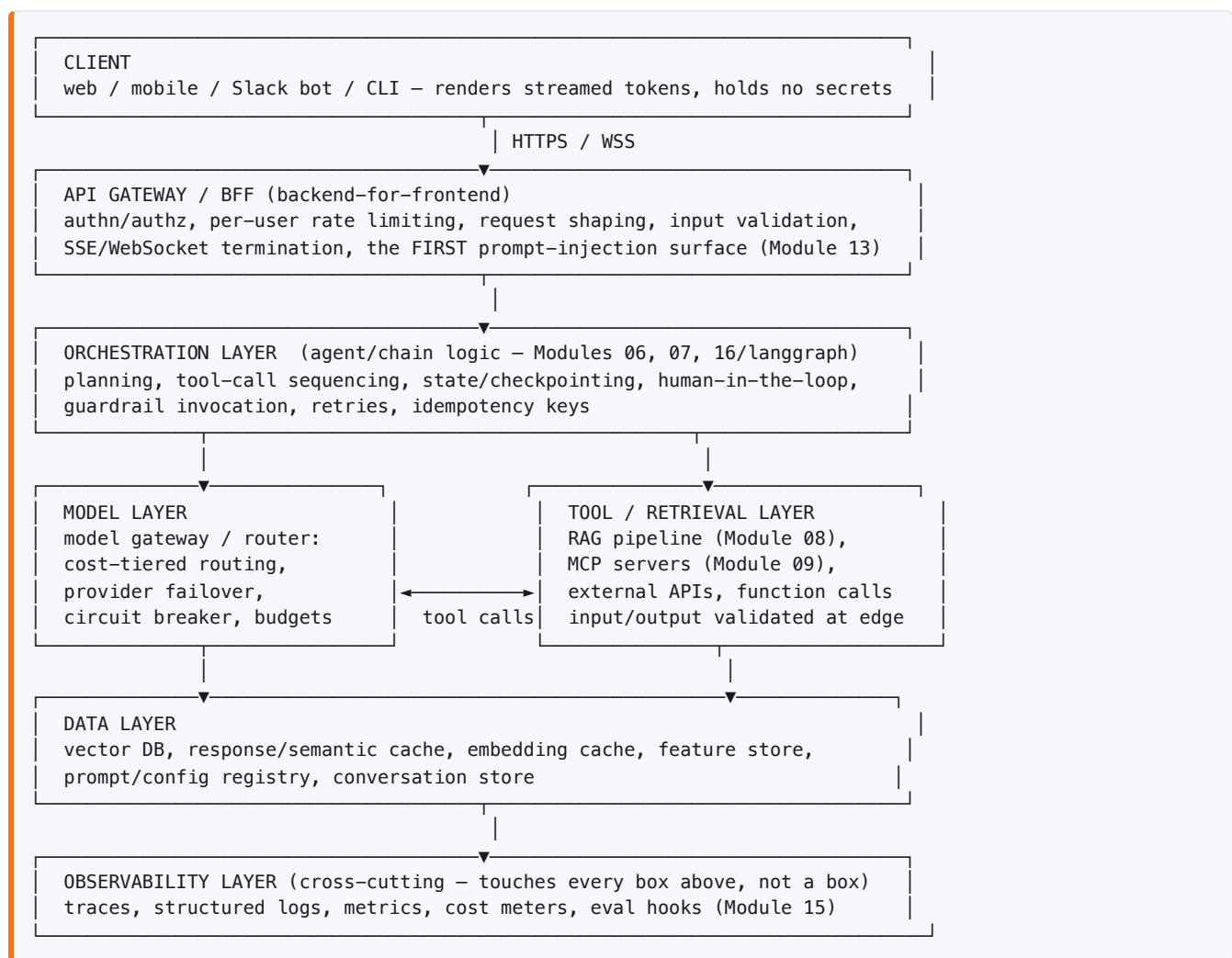
Modules 01–09 gave you the parts: a model (04), an agent loop (06), multi-agent orchestration (07), retrieval (08), a standard for wiring in tools (09). This note gives you the box you put them all in. Read it as an architecture review, not a tour — every layer exists because something went wrong when teams skipped it.

1. Why "architecture" is a separate skill from "model literacy"

A model call is a function: prompt in, tokens out. A *product* has to survive concurrent users, a flaky provider, a bad actor typing into the box, a compliance officer asking "who approved this answer," and a finance person asking why the bill tripled. None of that is solved by picking a better model. It's solved by where you put things — which is the definition of architecture.

The single biggest failure pattern in production LLM systems is not a bad model. It's a good model wired into a system with no cache, no fallback, no trace, and no budget ceiling. Everything in this module is a defense against that pattern.

2. The layered reference architecture



Observability is drawn last but it is not "last" architecturally — it is a horizontal concern that must instrument every other layer from day one. See [notes/02-production-patterns.md](#) [10-ai-architecture/notes/02-production-patterns.md] §2.

2.1 What each layer owns

Layer	Owns	Does NOT own
Client	rendering, optimistic UI, streaming display	auth secrets, business logic, prompt construction
API gateway / BFF	authn/authz, rate limits, input shape/size limits, SSE fan-out	model selection, retrieval logic
Orchestration	control flow, state, retries, tool sequencing, HITL gates	provider selection, caching
Model layer	provider abstraction, routing, failover, cost enforcement	knowing <i>why</i> a request needs a tool
Tool/retrieval	executing the RAG/MCP/API call correctly and safely	deciding <i>when</i> to call itself
Data layer	durable and cached state, freshness	request-level policy
Observability	seeing across all of the above	changing behavior at request time (mostly — budget breakers are the exception)

2.2 What leaks when this is done badly

Bad practice	What leaks	Symptom in production
Prompt construction happens in the client	prompt = attack surface, no server-side validation	prompt injection is trivial; you can't change the prompt without a mobile release
The orchestration layer calls providers directly	no single place to add a fallback	one provider outage takes the whole product down
Retrieval logic lives inside the agent's system prompt	you cannot swap the vector DB or reindex without a prompt change	RAG quality regressions get misdiagnosed as "the model got worse"
Auth checked only at the gateway, never re-checked before a tool call	a tool call authorized by session, not by the specific action	an agent with a stale session can act after permissions were revoked
Logging added after the first incident	no trace for the incident that taught you to add logging	you debug the <i>next</i> incident blind too, because half the surface still isn't instrumented
Cost limits enforced only in a nightly report	budget breach discovered a day late, not blocked at request time	a retry storm or a scraping bot produces a five-figure surprise bill

That table is the syllabus for the rest of this module — each bad row is fixed by a pattern below.

3. Model routing and gateway patterns

The **model gateway** is the single place that knows about providers, so nothing else has to. Put one component between your orchestration layer and every model call, even if you use only one provider today — retrofitting a gateway after three services hardcode a provider SDK is a much worse day.

3.1 What a gateway does

- **Provider abstraction** — one internal API (`gateway.complete(request)`), N provider SDKs behind it. Swapping OpenAI ↔ Anthropic ↔ a self-hosted vLLM endpoint touches one file, not every call site.
- **Cost-tiered routing** — classify the request (cheap heuristic: token count, a fast classifier, or a rule like "structured extraction always goes cheap") and route the easy 80% to a small/cheap model, escalating only what needs it. This is the biggest cost lever from Module 04 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md] — the gateway is where that policy actually lives in code.
- **Failover** — if the primary provider errors or times out, retry against a secondary automatically, inside the request, before the user notices.
- **Circuit breaking** — after N consecutive failures from a provider, stop calling it for a cooldown window instead of paying the timeout tax on every request. Half-open after cooldown: send one probe request; close the circuit again on success, reopen on failure.
- **Load balancing** — spread traffic across multiple deployments/regions of the same model for latency and to avoid a single rate limit ceiling.
- **Rate limits and budgets** — enforce a per-tenant and a global token/dollar budget *before* the call goes out, not in a report the next morning. A budget breaker is a circuit breaker keyed on spend, not failures.

`code/model_gateway_simulator.py` [10-ai-architecture/code/model_gateway_simulator.py] implements all five against simulated providers with distinct cost/latency/failure profiles, and prints a report comparing this to naive single-provider routing. Run it before you read further — the numbers are the argument.

3.2 Build vs. buy

	Build (your own gateway)	Buy (LiteLLM / a hosted router, e.g. AWS Bedrock, Portkey, OpenRouter)
Time to first routing rule	days, if you keep it small	hours — most support cost-tiering and failover out of the box
Provider coverage	whatever you write adapters for	broad, maintained by someone else, updated when providers change
Custom routing logic (your classifier, your business rules)	full control	usually pluggable, sometimes awkward to fit
Data residency / no-third-party-in-the-hot-path	trivial to guarantee	check the vendor's architecture — some proxy your tokens through their infra
Operational burden	you patch it, you're paged for it	vendor's problem, until they have an outage and now it's yours too

Opinion: start with LiteLLM (or your cloud's native router if you're single-cloud) as a thin proxy — it gives you the OpenAI-shaped interface, retries, and provider abstraction for free. Write your own gateway logic (cost tiers, budgets, business-specific routing) as a layer *in front of* that proxy, not instead of it. Build the whole thing yourself only when you have a routing requirement no gateway supports (e.g., routing on a proprietary classifier that must run in the same process for latency) or a hard requirement that no proxy vendor sits in the request path.

4. Caching strategies at every layer

Caching is the highest-leverage latency and cost optimization in this entire module, and the one most likely to silently corrupt correctness if you don't respect the trade-off. There are four distinct caches; conflating them is the most common design mistake.

Cache	Lives at	Hit condition	Win	Risk
Exact-match response cache	gateway / orchestration	byte-identical (normalized) request	near-zero latency, zero cost on hit	stale answer if underlying data changed
Semantic cache	gateway / orchestration	embedding-similarity above a threshold	catches paraphrases exact-match misses	returns a wrong cached answer for a similar-but-different question — see below
Prefix / prompt cache	provider (native feature)	shared byte-identical prefix	provider bills cached tokens far cheaper; no correctness risk	none, if the prefix is genuinely constant — see Module 04 §5 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md]
Embedding cache	retrieval layer	identical text was already embedded	skips a re-embed call during RAG ingestion or query time	correctness-safe; embeddings are deterministic per model version — but must be invalidated on embedding-model upgrade

4.1 The semantic cache correctness risk, concretely

A semantic cache stores `(query_embedding, response)` pairs and, on a new query, returns the cached response if cosine similarity to some stored query exceeds a threshold. This works beautifully for true paraphrases ("what's your return window" vs "how long do I have to return something"). It fails silently and expensively on **near-duplicate queries that differ in exactly the fact that matters**:

"What's the refund policy for orders under \$50?" vs. "What's the refund policy for orders under \$500?"

These two queries share 8 of 9 tokens. A bag-of-words or even a real sentence embedding will place them very close together — similarity in the 0.85–0.97 range depending on the model — while the correct answers are different policies. A semantic cache tuned for a high hit rate (loose threshold) will confidently return the *wrong* policy. `code/semantic_cache_correctness.py` [10-ai-architecture/code/semantic_cache_correctness.py] builds exactly this pair, sweeps the similarity threshold, and shows the false-hit numerically: at a loose threshold the cache reports a "hit" and returns the \$50 answer to the \$500 question; tightening the threshold (or adding a guard) fixes it at the cost of some legitimate cache hits. Run it — the number of percentage points you trade is more concrete than any rule of thumb.

Two fixes, in order of preference: 1. **Entity/number guard.** Extract high-stakes tokens (amounts, dates, IDs, negations) from the query with a cheap regex or NER pass; if they differ between the new query and the cached one, force a cache miss regardless of embedding similarity. Cheap, catches the sharpest class of failure.

2. **Tighten the threshold.** Reduces the false-hit rate but also reduces the legitimate hit rate on real paraphrases — there is no threshold that gets both for free, which is exactly the point of the demo script.

Never put a semantic cache in front of anything where a wrong answer is costly (pricing, medical, legal, refund policy, anything a user will act on). Put it in front of low-stakes, high-volume, tolerant-of-occasional-imprecision traffic (FAQ chat, casual Q&A) where an occasional near-miss is cheaper than the latency/cost of always calling the model.

4.2 Prompt/prefix caching — link, don't relitigate

Prefix caching is a provider-side mechanic, not an architectural cache you build — see Module 04 §5 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md] for the mechanism. The architectural implication for this module: **structure every prompt template so the stable part (system prompt, tool schemas, few-shot examples) comes first, and the variable part (user turn, retrieved context if it's genuinely per-request) comes last.** Get this ordering wrong at the orchestration layer and you pay full price for every request regardless of how good your provider's prefix cache is.

5. Data architecture for AI systems

5.1 The RAG ingestion pipeline, and the freshness problem

```
sources → crawl/sync → clean/normalize → chunk → embed → index (vector DB)
      |                                     |
      └─ track a content hash + last-modified per source ─┘
                (this is what makes re-indexing incremental, not a full rebuild)
```

The pipeline above is easy to build once and easy to leave stale forever. The hard problem isn't building it — it's the **sync/reindexing problem**: your source of truth (a wiki, a ticket system, a docs repo) keeps changing after you've indexed it, and a RAG system serving answers from a six-week-old index is worse than one that says "I don't know," because it's confidently wrong instead of visibly unhelpful.

Concrete freshness architecture: - **Content-hash-based incremental sync.** Store a hash per source document; a scheduled or webhook- triggered job re-crawls, diffs hashes, and only re-chunks/re-embeds/re-indexes what changed. Full rebuilds do not scale past a small corpus. - **Deletion handling.** A source document removed upstream must be removed from the index — this is the case teams forget, and it's how a RAG system cites a policy that no longer exists. **Tombstone or delete on next sync, not "eventually."** - **TTL by content type.** A support macro can go stale in days; a legal policy document might be authoritative for a year. Set per-source freshness SLAs and alert when a source hasn't synced within its SLA — this is an observability concern (§2 of notes/02-production-patterns.md [10-ai-architecture/notes/02-production-patterns.md]), not just a batch job. - **Version the index, not just the documents.** Keep the previous index queryable during a re-index so a bad ingestion run (bad chunking, a corrupted crawl) can be rolled back the same way you'd roll back a bad deploy — see the deployment section in the next note.

5.2 Vector database selection criteria

Do not pick a vector DB from a leaderboard. Pick it against your actual requirements:

Criterion	Why it matters	Questions to ask
Managed vs. self-hosted	ops burden vs. data residency/cost at scale	Who is paged when it's slow at 3 a.m.? Does data leave your VPC?
Hybrid search (vector + keyword/BM25)	pure vector search misses exact-term queries (SKUs, error codes, names)	Native hybrid, or do you fuse two systems yourself?
Metadata filtering	RAG almost always needs "search within tenant X / date range / doc type"	Is filtering pre- or post-vector-search? Pre-filter at scale is the hard engineering problem.
Scale (vectors, QPS, recall@latency)	a demo at 10k vectors tells you nothing about 50M	Ask for their recall-vs-latency curve at <i>your</i> vector count, not a marketing benchmark
Multi-tenancy	one index per tenant vs. shared index with metadata isolation	Namespace/collection isolation, or is tenant isolation your application's job?
Update/delete cost	freshness (\$5.1) depends on cheap upserts and deletes	Is a delete a soft-delete flag or a full segment rebuild?

Opinion: for most teams below ~10M vectors with a single cloud provider already chosen, use that provider's managed vector store or a managed vendor (Pinecone-class) and spend your engineering time on chunking and retrieval quality instead — retrieval quality has vastly more leverage on answer quality than which ANN index you picked. Self-host (pgvector on Postgres you already run, or Qdrant/Weaviate) when data residency requires it, when you're already running the infra to do it well, or when you're past the point a managed vendor's pricing curve makes sense.

5.3 The feature store pattern, mixed into the same system

Not every prediction in a modern AI product comes from an LLM. A production system routing support tickets might use a classical model (Module 01) for triage *and* an LLM for drafting the reply — both need consistent features (customer tier, account age, past ticket count) computed the same way online (at request time) and offline (at training time). A **feature store** is the layer that guarantees that consistency:

```
batch pipeline → offline feature store → training data (Module 01/02 pipelines)
|
└────────────────→ online feature store → low-latency lookup at request time (data layer, $2)
```

The failure mode a feature store prevents is **training/serving skew** — a feature computed one way in a nightly batch job and a subtly different way in the request-path code, so the model's online behavior doesn't match what it was evaluated on. If your system mixes classical ML and LLM calls (very common: a cheap classifier decides *whether* to invoke the expensive model at all — itself a cost-tiering pattern), the feature store belongs in the data layer next to the vector DB and the caches.

6. Forward links

Idea here	Where it returns
Cost-tiered routing	Module 04 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md] — the cost model this architecture executes
Circuit breakers, failover	notes/02-production-patterns.md [10-ai-architecture/notes/02-production-patterns.md] §1 — reliability patterns in depth
Prefix caching	Module 04 §5 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md] — the serving mechanic underneath
RAG ingestion, chunking, retrieval quality	Module 08 — RAG [10-ai-architecture/08-rag]
MCP servers as the tool layer	Module 09 — MCP [10-ai-architecture/09-mcp]
Agent state, checkpointing, the re-execution trap	Module 16 / langgraph [10-ai-architecture/16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]
Tracing this whole request path	Module 16 / langsmith [10-ai-architecture/16-ai-tech-stack/tracks/langsmith]
Where guardrails and input validation sit	notes/02-production-patterns.md [10-ai-architecture/notes/02-production-patterns.md] §3, hard link to Module 13 [10-ai-architecture/13-ai-security]
Auditability of prompt/model versions	Module 12 — Governance [10-ai-architecture/12-ai-governance]
Vector DB and feature quality vs. model choice	Module 01 [10-ai-architecture/01-ml-foundations/notes/01-core-concepts.md] §5 — features are still where projects are won